

- OPNsense et l'architecture réseau de mon lab
 - Introduction
 - Contexte professionnel
 - Technologies utilisées
 - Architecture du Réseau
 - Tableau du plan d'adressage IP
 - Tableau des spécifications réseau
 - Schéma de la topologie
 - Configuration des Équipements
 - 1. Interfaces WAN et LAN
 - 2. Accès GUI sécurisé
 - 3. NAT sortant
 - 4. Extension réseau — création des segments AD et DMZ par Terraform
 - 5. Rattachement des nouveaux segments à OPNsense
 - 6. DHCP sur le LAN
 - 7. DNS interne (Unbound) et transition vers Pi-hole
 - 8. Règles de pare-feu
 - 9. Exposition publique sans port-forward
 - Tests et Résultats
 - Difficultés Rencontrées et Solutions
 - Améliorations Possibles
 - Ce que ce Projet Démontre
 - Références et Ressources

OPNsense et l'architecture réseau de mon lab

Configuration détaillée d'OPNsense comme pare-feu/routeur de mon lab Proxmox, segmentation réseau en 4 zones, et gestion des bridges réseau par Terraform.

Auteur : Garlens Charles

Formation : BTS SIO – Option SISR

Année : 2025-2026

Introduction

OPNsense est le cœur réseau de tout mon lab : c'est lui qui route et filtre le trafic entre Internet et mes machines, et qui va bientôt séparer plusieurs zones isolées (services, futur lab Active Directory, future DMZ). Ce document détaille toute sa configuration, comment j'ai étendu le réseau à 4 segments avec Terraform, et les blocages réseau que j'ai rencontrés en le faisant.

Contexte professionnel

- **Défense en profondeur** : un pare-feu dédié en frontal, plutôt que de tout laisser à la box internet, c'est le premier réflexe de sécurité réseau qu'on attend en entreprise.
- **Microsegmentation** : séparer LAN/AD/DMZ en zones isolées limite ce qu'un attaquant peut atteindre s'il compromet une seule machine — un principe qu'on retrouve dans toute architecture réseau sérieuse.
- **Principe du moindre privilège réseau** : chaque nouvelle interface OPNsense bloque tout le trafic par défaut ; on n'ouvre que ce qui est explicitement nécessaire.
- **Exposition publique maîtrisée** : pas de port-forward ouvert sur ma box, l'exposition se fait par tunnel sortant — zéro surface d'attaque entrante depuis l'extérieur.
- **Documentation pour l'audit** : chaque réglage réseau est noté avec sa raison, pas juste appliqué au hasard — utile si je dois un jour justifier une configuration en entretien ou en soutenance.

Technologies utilisées

- OPNsense

— pare-feu/routeur open source, basé sur FreeBSD/pf

- Terraform

(provider bpg/proxmox) — création des bridges réseau en Infrastructure as Code

- Unbound DNS

— résolveur DNS intégré à OPNsense

- ISC DHCPv4

— service DHCP pour les clients dynamiques du LAN

Architecture du Réseau

Mon lab est structuré en 4 segments réseau, tous routés et filtrés par OPNsense. Un seul segment (LAN) est actif avec de vrais services aujourd'hui ; les deux autres (AD, DMZ) ont leurs bridges créés mais attendent encore leurs machines.

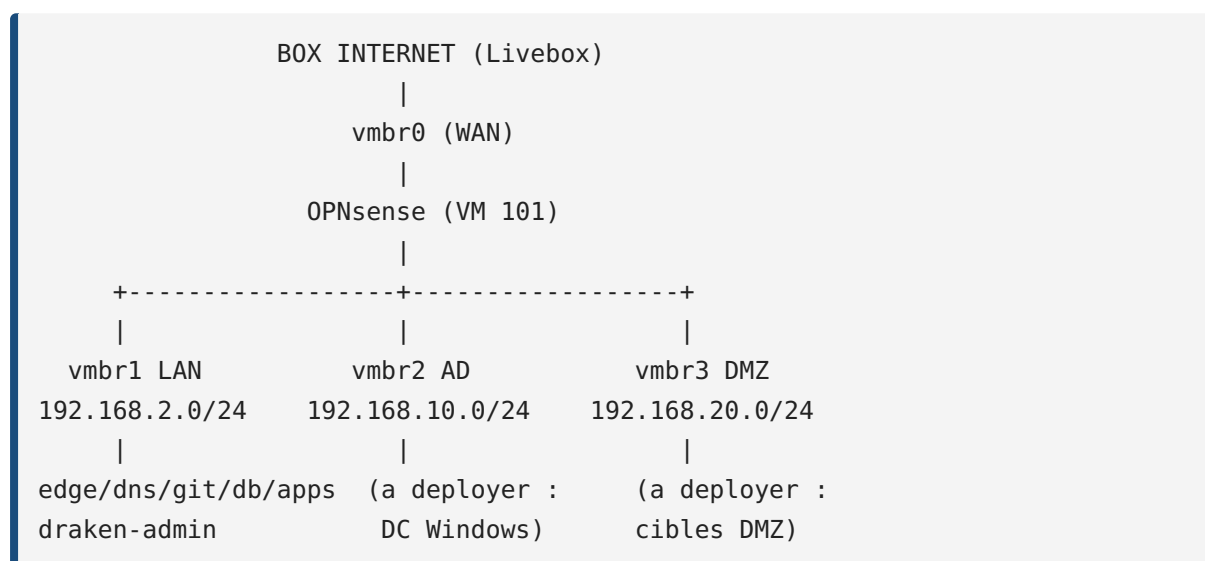
Tableau du plan d'adressage IP

Équipement	Adresse IP	Rôle
OPNsense (WAN)	DHCP (Livebox)	Interface Internet
OPNsense (LAN)	192.168.2.1	Gateway du réseau lab principal
OPNsense (AD)	192.168.10.1	Gateway du futur lab Active Directory
OPNsense (DMZ)	192.168.20.1	Gateway de la future zone démilitarisée
draken-admin (CT 102)	192.168.2.101	Bastion IaC
edge (CT 110)	192.168.2.10	Reverse proxy
dns (CT 111)	192.168.2.11	Pi-hole
git (CT 112)	192.168.2.12	Gitea
db (CT 113)	192.168.2.13	PostgreSQL
apps (VM 120)	192.168.2.20	Docker

Tableau des spécifications réseau

Ressource	Valeur
Segments réseau	4 (WAN, LAN, AD, DMZ)
Bridges Proxmox	vmbr0 (WAN, carte physique), vmbr1/2/3 (internes, sans carte physique)
Plage DHCP LAN	192.168.2.100 → .199
Résolveur DNS	Unbound (OPNsense) → transition vers Pi-hole

Schéma de la topologie



Configuration des Équipements

1. Interfaces WAN et LAN

Pourquoi cette étape est importante ? C'est la base absolue : sans cette étape, OPNsense n'a même pas de réseau à router. J'ai aussi dû comprendre une subtilité liée au double-NAT de ma box internet.

```
Interfaces → Assignments : WAN = vtnet0 (net0 → vmbri0), LAN = vtnet1 (net1 → vmbri1)
Interfaces → WAN : IPv4 DHCP
Interfaces → LAN : IPv4 Static, 192.168.2.1/24
```

Sur l'interface WAN, j'ai dû décocher « Block private networks » et « Block bogon networks » : ma box internet fait déjà un premier NAT, donc l'IP que reçoit OPNsense côté WAN est elle-même une IP privée. Sans décocher ces options, OPNsense bloque son propre trafic en pensant qu'il s'agit d'une usurpation.

Résultat : OPNsense joignable sur

```
192.168.2.1
```

, connecté à Internet côté WAN.

2. Accès GUI sécurisé

Pourquoi cette étape est importante ? L'interface d'administration d'un pare-feu est une cible évidente ; autant la protéger correctement dès le début plutôt que d'attendre un incident.

```
System → Settings → Administration : Protocol HTTPS
Décocher "Disable web GUI redirect rule" → http:// redirige vers https://
```

Résultat : GUI accessible uniquement en HTTPS (

```
https://192.168.2.1
```

), HTTP renvoie un 301 vers HTTPS.

3. NAT sortant

Pourquoi cette étape est importante ? Sans NAT sortant configuré, mes machines internes (adressage privé 192.168.2.0/24) ne peuvent tout simplement pas sortir sur Internet — c'est ce qui traduit leurs adresses privées vers l'IP publique de la box.

```
Firewall → NAT → Outbound : mode Automatic
```

Résultat : tout le lab a accès à Internet via OPNsense.

4. Extension réseau — création des segments AD et DMZ par Terraform

Pourquoi cette étape est importante ? Je voulais que même le réseau (pas juste les VM) soit reproductible et versionné. Ça m'a aussi appris que Terraform peut gérer bien plus que des machines virtuelles.

```
# modules/network/main.tf
resource "proxmox_virtual_environment_network_linux_bridge" "ad" {
  node_name = var.node_name
  name      = "vmbr2"
  comment   = "Lab AD Windows (isole, sans carte physique)"
  ports     = []
  autostart = true
}

resource "proxmox_virtual_environment_network_linux_bridge" "dmz" {
  node_name = var.node_name
  name      = "vmbr3"
  comment   = "DMZ - services exposes / vulnérables"
  ports     = []
  autostart = true
}
```

```
cd ~/draken-infra/terraform
terraform apply
ssh root@192.168.1.50 "ip a | grep -E 'vmbr2|vmbr3'"
```

J'ai volontairement laissé

```
vmbr0
```

/

```
vmbr1
```

hors de ce module : ils existaient déjà avant que je commence à utiliser Terraform sur ce projet, et un import risqué aurait pu provoquer un drift de state destructeur sur un lab qui doit rester joignable.

Résultat vérifié :

```
vmbr2
```

et

```
vmbr3
```

actifs sur le nœud après

```
terraform apply
```

5. Rattachement des nouveaux segments à OPNsense

Pourquoi cette étape est importante ? Ça a été mon blocage réseau le plus long : comprendre qu'un redémarrage logiciel ne suffit pas toujours à faire reconnaître du nouveau matériel virtuel.

```
qm set 101 -net2 virtio,bridge=vmbr2
qm set 101 -net3 virtio,bridge=vmbr3
qm reboot 101
# → net2/net3 n'apparaissent PAS dans Interfaces → Assignments malgré le
    reboot

qm stop 101
qm start 101
# → net2/net3 apparaissent enfin
```

```
Interfaces → Assignments : vtnet2 → OPT1 (AD), vtnet3 → OPT2 (DMZ), activer
les deux
OPT1 (AD) : IPv4 Static 192.168.10.1/24
OPT2 (DMZ) : IPv4 Static 192.168.20.1/24
```

Résultat : les 2 nouveaux segments sont routés par OPNsense, avec leur propre gateway.

6. DHCP sur le LAN

Pourquoi cette étape est importante ? Mes services critiques ont une IP statique fixée par Terraform et n'ont pas besoin de DHCP, mais je voulais quand même une plage pour des clients de test ou des conteneurs jetables.

```
Services → ISC DHCPv4 → LAN : Enable, range 192.168.2.100 → 192.168.2.199
```

Résultat : les machines qui ne sont pas gérées par Terraform (postes de test, clients de passage) reçoivent automatiquement une IP.

7. DNS interne (Unbound) et transition vers Pi-hole

Pourquoi cette étape est importante ? Je voulais qu'à terme mon filtrage DNS (Pi-hole) devienne le résolveur principal du lab, sans perdre la résolution pendant la transition.

```
Services → Unbound DNS : Enable, listen sur LAN
```

Une fois le LXC

```
dns
```

(Pi-hole,

```
.11
```

) opérationnel, j'ai basculé le DNS distribué par DHCP :

```
Services → ISC DHCPv4 → LAN : DNS servers = 192.168.2.11
```

Résultat : Pi-hole devient le résolveur principal du lab, OPNsense (Unbound) reste en résolveur de secours.

8. Règles de pare-feu

Pourquoi cette étape est importante ? Sans règle explicite, OPNsense bloque tout par défaut sur une interface — c'est volontaire (sécurité par défaut), mais ça veut dire qu'il faut explicitement autoriser ce dont j'ai besoin.

```
Firewall → Rules → LAN : règle "LAN net → any" (allow) – nécessaire pour que le lab sorte sur Internet
```

Résultat : le trafic sortant du LAN fonctionne ; les interfaces AD et DMZ, elles, bloquent tout par défaut tant que je n'y ai pas ajouté de règle explicite — exactement ce que je veux avant d'y placer des machines sensibles.

9. Exposition publique sans port-forward

Pourquoi cette étape est importante ? Je ne voulais ouvrir aucun port entrant sur ma box internet — un choix de sécurité qui évite complètement la surface d'attaque classique d'un serveur exposé.

Pas de configuration NAT entrante sur OPNsense : l'exposition future du portfolio se fera via un tunnel sortant (Cloudflare Tunnel) depuis le LXC

edge

, jamais par ouverture de port. L'administration à distance passe uniquement par Tailscale, jamais par la GUI OPNsense exposée sur le WAN.

Résultat : zéro port ouvert entrant sur la box, aucune surface d'attaque directe depuis Internet.

Tests et Résultats

Test	Commande	Résultat
GUI HTTPS active	<pre>curl -Ik https://192.168.2.1</pre>	200
Redirection HTTP → HTTPS	<pre>curl -I http://192.168.2.1</pre>	301
Gateway LAN joignable	<pre>ping 192.168.2.1</pre> depuis le lab	OK
Internet depuis le lab	<pre>curl -I https://google.com</pre> depuis une machine du LAN	200, NAT sortant fonctionnel
Bridges après terraform apply	<pre>ip a \ grep vmbr</pre>	puis après <pre>qm stop</pre> / <pre>qm start</pre>
Interfaces AD/DMZ visibles	Interfaces → Assignments (GUI)	vtnet2/vtnet3 assignées après redémarrage complet
Blocage par défaut sur AD/DMZ	tentative de connexion depuis AD vers LAN sans règle	bloqué (comportement attendu, pas encore de règle d'autorisation)

Difficultés Rencontrées et Solutions

#	Problème	Cause	Solution
1	Trafic WAN bloqué alors que la connexion internet fonctionnait	Ma box fait déjà un NAT ; OPNsense recevait une IP privée côté WAN et la traitait comme suspecte	Décocher “Block private networks” et “Block bogon networks” sur l’interface WAN
2	<pre>net2 / net3</pre> invisibles dans OPNsense après <pre>qm reboot</pre>	Un redémarrage logiciel ne force pas toujours la réénumération du bus PCI virtuel	<pre>qm stop 101</pre> puis <pre>qm start 101</pre> (arrêt complet)
3	Hypothèse de départ erronée : Terraform ne générerait que des VM/LXC, pas le réseau	Méconnaissance du provider bpg	Documentation officielle confirmant la ressource <pre>network_linux_bridge</pre>
4	Risque si <pre>vmbr0</pre> / <pre>vmbr1</pre> étaient importés dans le state Terraform	Drift de state possible → recréation destructrice sur un lab qui doit rester joignable	Terraform ne gère que le nouveau (<pre>vmbr2</pre> / <pre>vmbr3</pre>), le reste en gestion manuelle

Améliorations Possibles

Sécurité renforcée - Ajouter des règles de pare-feu explicites d’isolation stricte

AD → LAN

et

DMZ → LAN

avant de placer de vraies machines sur ces segments - Créer un utilisateur admin dédié sur OPNsense plutôt que d'utiliser uniquement

root

- Exporter et versionner la configuration OPNsense (

config.xml

) dans Gitea, pour une restauration en un clic

Infrastructure étendue - Déployer les VM du lab AD (contrôleur de domaine Windows + client) par clonage Terraform d'un template sysprep sur le segment

vmbr2

- Ajouter des cibles volontairement vulnérables sur le segment

vmbr3

(DMZ) pour des exercices d'intrusion isolés du reste du lab - Ajouter des réservations DHCP par adresse MAC pour que mes services apparaissent nommés dans l'interface OPNsense

Supervision réseau - Activer les logs de pare-feu détaillés sur les interfaces AD/DMZ une fois des machines en place, pour repérer toute tentative de sortie non autorisée - Ajouter un dashboard de trafic réseau (débit, connexions actives) en complément de la supervision applicative déjà en place

Ce que ce Projet Démontre

Compétence	Description
Pare-feu et routage	Configuration complète d'OPNsense (interfaces, NAT, DHCP, DNS, règles)
Segmentation réseau	Architecture 4 zones (WAN/LAN/AD/DMZ), blocage par défaut sur les nouvelles interfaces
Infrastructure as Code réseau	Création de bridges réseau versionnés via Terraform, pas seulement des VM
Diagnostic réseau bas niveau	Identification qu'un reboot logiciel ne suffit pas à une rénumération PCI virtuelle
Sécurité par conception	Exposition publique sans port-forward, administration distante uniquement via VPN

Références et Ressources

- Documentation officielle OPNsense (interfaces, NAT, règles de pare-feu, Unbound DNS)
- Documentation du provider Terraform bpg/proxmox (

```
network_linux_bridge
```

)
- Documentation Proxmox VE (gestion des bridges réseau,

```
qm
```

)
- RFC 1918 (adressage IP privé)